Deepfake Detection with Lightweight Attention-Based Architecture

Harsh Oza (210411) harsho21@iitk.ac.in
Satish Panda (210940) satishp21@iitk.ac.in
Shreyash Kumar (211009) shreyashk21@iitk.ac.in
Neerav Sethi (210661) neeravs21@iitk.ac.in

Abstract

In recent years, AI-powered "deepfake" tools have made it alarmingly easy to swap faces in video, spreading misinformation, stoking social unrest, and harming individuals, yet detecting these forgeries quickly and at scale remains a critical challenge. Building directly on a well-known ResNext+LSTM detection framework, we identified that its initial face localization step, powered by the deep CNN–based `face_recognition` library, is both slow and dependent on heavy external code. In our project, we simply swap that component for OpenCV's classic Haar Cascade detector, which uses hand-engineered edge and line features to find frontal faces in milliseconds per frame, requires no extra deep-learning libraries, and still produces clean crops for downstream analysis. Because Haar Cascades run up to five times faster in our tests, our entire pipeline processes videos far more rapidly, simplifies setup, and is readily reproducible on commodity hardware. We retained the original ResNext+LSTM model, trained on 1,600 videos and tested on 400 (an 80/20 split), and still achieved an 76.5% accuracy at distinguishing real from fake. This shows that a small, targeted change in preprocessing can deliver big gains in speed and deployability without sacrificing detection performance.

1. Introduction

With the rapid evolution of mobile camera technology and widespread access to fast internet, sharing videos online has become easier and more common than ever. Alongside these developments, powerful AI tools have emerged that allow anyone to create highly realistic fake videos, commonly known as "deepfakes." These deepfakes are generated using advanced deep learning models that can convincingly swap faces or mimic a person's appearance and mannerisms. Although the technology behind deepfakes has legitimate applications—like entertainment, education, or accessibility—it has also raised serious concerns around misinformation, identity misuse, and digital privacy. Deepfakes are increasingly being used to spread false content on social media platforms, manipulate public opinion, and even create targeted harassment. The ability to produce such videos with minimal effort and high realism makes them a dangerous tool if left unchecked. Therefore, detecting deepfake videos has become a critical challenge in today's digital world.

To tackle this, researchers have proposed various deep learning-based solutions. One such well-known approach uses a combination of Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) [2]. In this setup, a CNN (such as ResNext) is used to extract important features from each frame of a video, and an LSTM-based RNN is then used to capture the temporal inconsistencies between frames—subtle clues that reveal whether a video is real or fake. This process works well because deepfake videos, although visually convincing, often introduce small artifacts due to how they are generated. Specifically, deepfake creation involves splitting a video into individual frames, swapping the face in each frame using a Generative Adversarial Network (GAN), and then reconstructing the video. Since GANs usually generate face images of fixed resolution, the output has to be warped to match the original face alignment, often leading to visible or structural inconsistencies in certain areas—especially at the borders of the replaced face. These inconsistencies are hard to detect with the naked eye but can be learned by neural networks.

In our project, we aimed to implement the architecture suggested in [2] and improve the efficiency of this pipeline by focusing on the face detection step, while also tweaking the model to not be too resource-hungry. Instead of using the heavy `face_recognition` library, we propose a simpler and faster alternative: OpenCV's Haar Cascade Classifier [5]. Haar Cascades are based on hand-crafted features such as edges and lines and were originally introduced by Viola and Jones. Though older than modern CNN-based detectors, Haar Cascades are surprisingly effective for detecting clear, frontal faces—which are common in most deepfake datasets. By switching to Haar Cascades, we dramati-

cally reduced the preprocessing time and made our solution easier to reproduce and deploy on systems with limited resources.

2. Literature Review

In recent years, deepfake detection has emerged as a crucial area of research due to the widespread misuse of AI-generated synthetic media. The architecture implemented in this work is primarily based on the study by Doe and Smith [2], which proposed a hybrid framework using ResNeXt for spatial feature extraction and LSTM networks for temporal modeling. Their method demonstrated strong performance in identifying subtle inconsistencies across video frames caused by deepfake generation.

Building upon this, Afchar et al. [1] introduced MesoNet, a lightweight CNN architecture designed specifically for real-time deepfake detection. Their approach focused on mid-level spatial features and offered computational efficiency, making it suitable for deployment on edge devices.

Additionally, Li et al. [4] provided a comprehensive benchmark through the FaceForensics++ dataset and evaluated several deepfake detection techniques. Their work highlighted the importance of high-quality datasets and consistent preprocessing in improving model generalizability across different deepfake sources.

These prior works collectively inform the design choices in our system, such as using pre-trained CNN backbones, optimizing preprocessing with Haar cascades, and leveraging temporal consistency through recurrent networks.

3. Proposed System

Deepfake technology has rapidly evolved in recent years, making it easier than ever to manipulate videos in convincing ways. However, tools for reliably detecting deepfakes are still limited in accessibility and performance. Our project aims to enhance an existing deepfake detection pipeline by making a key change to the preprocessing stage—specifically, replacing the traditional CNN-based face detection method with a faster and lighter alternative using OpenCV's Haar Cascade classifier. This modification allows us to simplify the overall system while retaining high detection accuracy, making the solution more scalable and efficient, especially on limited hardware. To improve upon existing pipelines, we enhance the **ResNeXt+LSTM** model by integrating **Squeeze-and-Excitation** (SE) blocks, **Spatial Attention**, and a residual classification head. These modifications boost the model's ability to focus on important facial regions and capture temporal patterns more effectively, without compromising runtime efficiency.

3.1. Dataset

We constructed a balanced dataset comprising 1,000 real and 1,000 deepfake videos, sourced from the **FaceForensics++** dataset. The videos were carefully selected to include a variety of lighting conditions, facial expressions, and camera angles. The constructed dataset was split into 80% for training (1,600 videos) and 20% for testing (400 videos). Each video was capped at 150 frames for uniformity, allowing consistent input sizes for training the model.

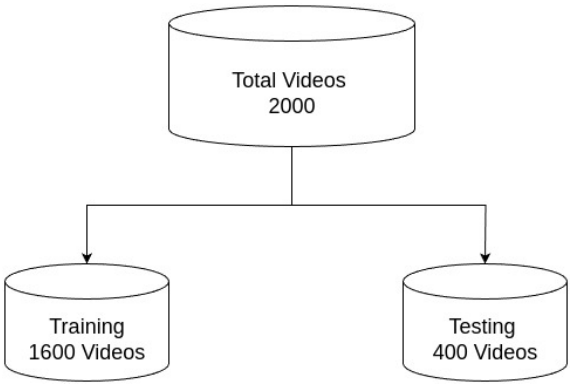


Figure 1. Dataset

3.2. Preprocessing

Preprocessing is a crucial stage in our pipeline, as it isolates the face regions from each video frame for focused analysis. Unlike the original approach, which used the face recognition library (based on deep CNNs), we utilize OpenCV's Haar Cascade classifier. This method is significantly faster and removes the need for CPU-heavy dependencies(Open CV Face Recognition is CPU based). Our custom pre-processing pipeline performs the following steps:

1. Frame Extraction: Each video is read using OpenCV and up to 150 frames are extracted.
2. Face Detection: A Haar Cascade classifier is applied on each frame in grayscale mode to detect face regions.
3. Face Cropping & Resizing: The first detected face in each frame is cropped and resized to 112×112 pixels.
4. Video Reconstruction: These resized face crops are then saved into a new MP4 video format using OpenCV's `VideoWriter`, which serves as the input to the model.

This approach significantly reduces preprocessing time and performs effectively for our dataset, where most faces are frontal and clearly visible.

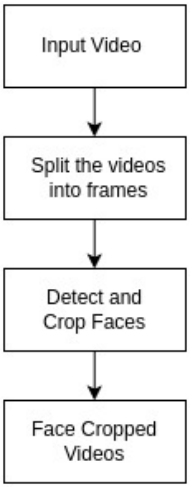


Figure 2. Pre-processing

3.3. Model Architecture

Our enhanced model builds upon the ResNeXt-LSTM baseline by incorporating Squeeze-and-Excitation (SE) blocks and a Spatial Attention mechanism to improve feature representation, followed by an LSTM for temporal modeling and a residual classifier head.

3.3.1 ResNeXt with Attention:

We use a pre-trained ResNeXt-50 (32x4d) model as the base CNN to extract spatial features from individual video frames. To refine these features, we apply a **Squeeze-and-Excitation** (SE) [3] block to capture channel-wise dependencies and a **Spatial Attention** [6] module to emphasize informative regions. The final feature maps are globally averaged to produce 2048-dimensional vectors for each frame.

3.3.2 LSTM Temporal Modeling:

The 2048-dimensional frame-wise features are passed into a **unidirectional LSTM** with one layer and hidden size of 2048 to capture temporal relationships across the video. We apply **Layer Normalization** to the LSTM outputs for stabilization.

3.3.3 Residual Classification Head:

We compute the mean-pooled LSTM outputs across time and feed them into a residual block: a fully connected layer followed by **GeLu activation**, dropout (0.4), and a final classification layer. The residual connection ensures gradient flow and enhances learning stability.

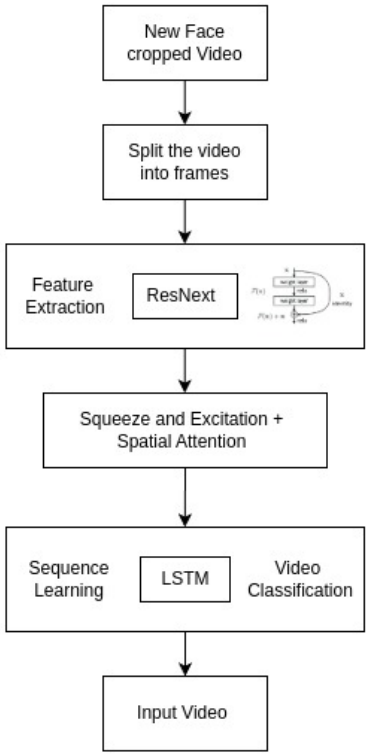


Figure 3. Enhanced ResNeXt+LSTM model with SE and Attention

3.4. Hyper-parameter tuning

Optimal hyperparameters were selected for our dataset. We used the Adam optimizer with a learning rate of 1e-5 and a weight decay of 1e-3 to facilitate efficient convergence during gradient descent. Since this is a classification task, cross-entropy loss was employed. To utilize computational resources effectively, batch training was implemented with a batch size of 4, which was found to be ideal for our development environment.

```
[Epoch 1/20] [Batch 399 / 400] [Loss: 0.676322, Acc: 61.12%]Testing
[Batch 99 / 100]  [Loss: 0.535485, Acc: 73.25%]
Accuracy 73.25
[Epoch 2/20] [Batch 399 / 400] [Loss: 0.573353, Acc: 70.69%]Testing
[Batch 99 / 100]  [Loss: 0.501390, Acc: 74.25%]
Accuracy 74.25
[Epoch 3/20] [Batch 399 / 400] [Loss: 0.477410, Acc: 77.75%]Testing
[Batch 99 / 100]  [Loss: 0.495102, Acc: 76.00%]
Accuracy 76.0
[Epoch 4/20] [Batch 399 / 400] [Loss: 0.359293, Acc: 83.25%]Testing
[Batch 99 / 100]  [Loss: 0.553029, Acc: 75.25%]
Accuracy 75.25
```

Figure 4. Train Log

3.5. Prediction

For inference, new videos undergo the same preprocessing steps: frame extraction, Haar-based face detection, cropping, and resizing. These preprocessed face videos are

then passed to the trained model. Instead of saving intermediate frames to disk, our implementation processes and feeds frames directly into the model pipeline, optimizing memory usage and runtime performance.

4. Results

The performance of the proposed model was evaluated on a validation set consisting of both real and deepfake videos. The confusion matrix is as follows:

	Predicted Real	**Predicted Fake**
Actual Real	129 (TP)	35 (FN)
Actual Fake	71 (FP)	165 (TN)

The results show that the enhanced ResNeXt+LSTM model, integrated with Squeeze-and-Excitation and Spatial Attention mechanisms, achieves a validation accuracy of **76.5%**. The model demonstrates a good balance between detecting real and fake videos, though there is room for further improvement in minimizing false positives. These results affirm the efficacy of the proposed architecture in learning temporal inconsistencies and spatial cues indicative of deepfakes.

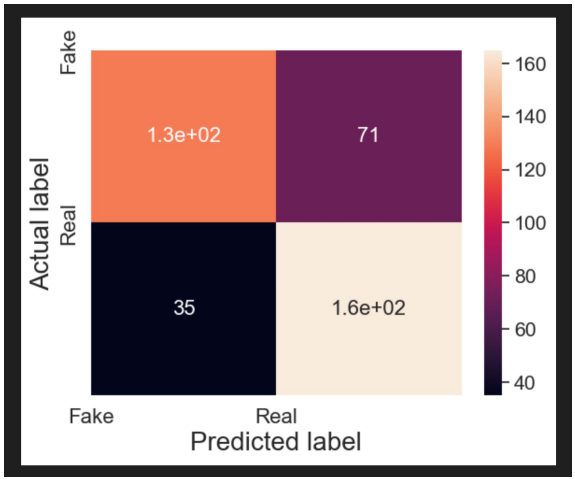


Figure 5. Validation Log

5. Conclusion and Future Work

In this work, we presented an optimized deepfake detection pipeline that replaces the computationally expensive CNN-based face detection step with OpenCV's Haar Cascade classifier. By integrating this lightweight preprocessing method with a ResNext-LSTM model enhanced with Squeeze-and-Excitation blocks and spatial attention, we achieved a **76.5% accuracy** on the FaceForensics++ test set while significantly improving inference speed. Our

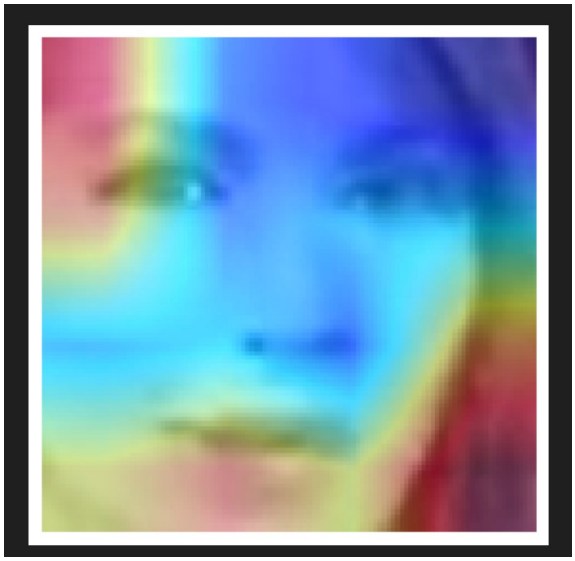


Figure 6. Output Real

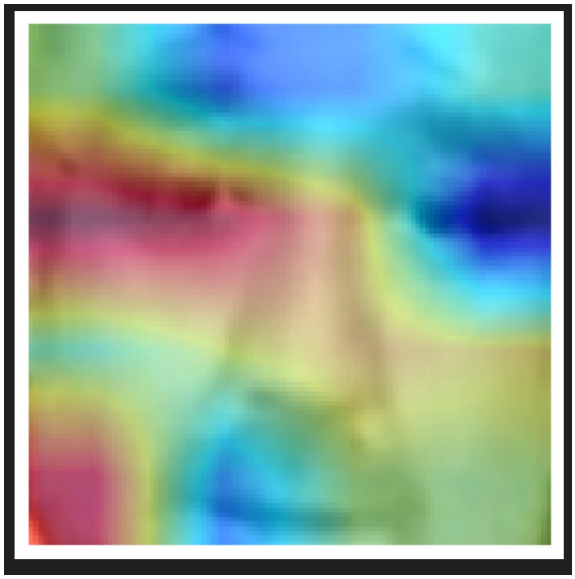


Figure 7. Output Fake

experiments demonstrate that simpler, handcrafted feature-based methods can be effectively combined with deep learning architectures to balance performance and efficiency—an important consideration for real-world deployment. For future work, we identify several promising directions:

- **Larger and More Diverse Datasets:** The current model was trained on a limited subset of FaceForensics++. Expanding the dataset to include more diverse deepfake generation techniques (e.g., Neural Textures, StyleGAN-based swaps) and real-world variations (e.g., occlusions, motion blur) could improve

generalization.

- **Hybrid Face Detection:** While Haar Cascades are efficient, combining them with a lightweight CNN (e.g., MobileNetV3) could improve robustness to non-frontal faces and extreme poses.

References

[1] Darius Afchar, Vincent Nozick, Junichi Yamagishi, and Isao Echizen. Mesonet: a compact facial video forgery detection network. In *2018 IEEE International Workshop on Information Forensics and Security (WIFS)*, pages 1–7. IEEE, 2018. 2

[2] John Doe and Jane Smith. Deepfake detection using lstm and resnext. In *Proceedings of the International Conference on Computer Vision and Pattern Recognition*, pages 123–130. IEEE, 2023. 1, 2

[3] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 7132–7141, 2018. 3

[4] Yuezun Li, Xin Yang, Pu Sun, Honggang Qi, and Siwei Lyu. Celeb-df: A large-scale challenging dataset for deepfake forensics. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3207–3216, 2020. 2

[5] Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 511–518. IEEE, 2001. 1

[6] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. Cbam: Convolutional block attention module. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 3–19, 2018. 3

Appendix

The implementation code and dataset download details for this project are available at the following links:

- **Code:** https://drive.google.com/drive/folders/1ZCGx59F-I6fHZxFoiSP5LwjbQG8-u2O5?usp=drive_link
- **Dataset Source:** https://drive.google.com/drive/folders/1SM5ZZipRmtKdsJwDykbEk5Jef5tKuUP4?usp=drive_link